

Krama: Participant-Centric Consensus for Blockchains

ANONYMOUS AUTHOR(S)

1 Introduction

Blockchain systems starting with Bitcoin [Nak09], democratized the ability to maintain a global and shared distributed ledger of transactions without the need for a trusted third party. Ethereum [But15] further built on this vision, building an open-access general purpose decentralised application platform - a functionality which most modern blockchains provide. In the modern world, human interactions are becoming increasingly digital. Such interactions are highly contextual - for instance, an interaction where a person pays for a coffee is different from the person buying real estate although both involve a money transfer. Therefore, a network for such digital human interactions must integrate their contextual nature. However, in today's general purpose smart contract platforms, the state intelligence of participants for each application is siloed with the contracts themselves. This results in inflexibility as the intelligence to seamlessly cater to the largely context-sensitive real world interactions is not available. For instance, in the example interactions given earlier, the contextual difference between the two cannot be leveraged in traditional blockchains to incorporate more trust in the latter interaction due to the siloed nature of intelligence. If the contextual intelligence of a participant is leveraged across all applications, a participant-centric network is created whose functionality powers the digitally interacting future for humans. This requires a fundamental infrastructure shift, moving the intelligence from applications hosted on smart contracts to the network itself. This intelligence in the network is anchored to the participants, making participant preferences at the centre of applications built on the network. This is captured by the *context object* (analogous to state) of participants, having four dimensions of Value, Logic, Files, and Trust.

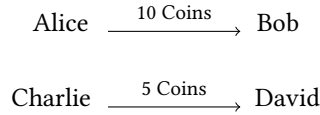


Fig. 1. Interactions between Alice, Bob, Charlie, and David

Consider 4 distinct participants, Alice, Bob, Charlie, and David, who interact with each other in a blockchain system. Together, they submit two distinct transactions as given in Figure 1 - Alice sends 10 coins to Bob and Charlie sends 5 coins to David. In a traditional blockchain, validators that receive both these transactions ignore the context of the interacting participants and communicate with all other validators to decide on the order of the transactions. Here, the consensus mechanism is participant agnostic and only concerns the global state.

In this work, we present the Interaction Finality Protocol (IFP) - a consensus protocol powering participant-centric blockchains which fundamentally integrates the participant into the consensus mechanism. IFP achieves consensus on updated context objects of participants involved in interactions. It powers a blockchain where each participant has its respective separate chains that intersect only when an *interaction* (analogous to transaction) occurs between the participant. Here, the current context object of a participant can be found in the head of its respective chain. This

decouples consensus, providing the ability to integrate participants' trust preferences into the consensus mechanism. However, when multiple concurrently occurring interactions involve a common participant, it is crucial to ensure that they are ordered globally among all other concurrent interactions.

To achieve this, IFP's consensus mechanism associates a committee of validators with each participant. Every interaction forms a unique Interaction Consensus Set (ICS) consisting of the committees of the interacting participants, along with a randomly chosen set of validators from the network that order the interaction together. The ICS ensures that the validators maintaining the participant's chains order the interaction globally among all other concurrent interactions. The randomly chosen set of validators provides the power of the entire network, having to corrupt a significant portion of the entire network to affect IFP's safety. Further, this ability to concurrently process interactions can lead to significant performance improvements in terms of throughput.

1.1 Organisation

In what follows, we first present the preliminary definitions in Section 2 and the system model in Section 3. We then present the core KRAMA protocol in Section 4 followed by its analysis in Section 5. Finally, we discuss related work in Section 6 and conclude in Section 7.

2 Preliminaries

Two actors drive the algorithm in KRAMA. First, validators (denoted by $\mathbb{V}$), that run the protocol to reach consensus. Second, Participants ($\mathbb{P}$) that submit *interactions* between each other to the validators. Each participant $p \in \mathbb{P}$ defines a fixed-size set of validators $C_p \subseteq \mathbb{V}$ ($|C_p| = n_c$) that mandatorily participate when achieving consensus for interactions involving the participant p . We will refer to this set of validators as its *context*. The context of each participant is public and changes periodically.

The state of each participant (capturing the assets of the participant) is stored by each validator. Formally, we define the state of each participant as follows.

State. The *state* of a participant p is defined as $S_p = (l_p, C_p)$ where l_p is the valuation of its local variables and C_p its context as defined earlier. Subsequently, $\mathbb{S}_p$ denotes the set of possible states for a participant p . We define the *global state* of the network as a union of the state of all participants, $\mathcal{S} = \bigcup_{p \in \mathbb{P}} S_p$, and $\mathbb{S}$ is the set of all possible global states.

Interaction. The state of each participant (and consequently the global state) is updated through *interactions* (analogous to transactions). Participants send interactions to the network which are processed by the validators.

Formally, we define an interaction I between a set of participants P_I as a set of transition functions $\delta_I^p : \mathbb{S}_p \rightarrow \mathbb{S}_p$ for each $p \in P_I$. An interaction abstracts the semantics of the underlying transitions between the states of the participants involved. Naturally, each validator updates the state of all participants $p \in P_I$ by applying the corresponding transition function $S'_p = \delta_I^p(S_p)$.

Tesseract. From a given global state S and an interaction I , a *Tesseract* T represents the set of updated states for each participant involved in the interaction. Formally, $T = \{\delta_I^p(S_p) \mid p \in P_I\}$ and $\mathbb{T}$ is the set of all possible Tesseracts. Validators internally log the Tesseract for each interaction and the log serves as snapshots of the global state. The log is stored using a data structure called Multi-Linked DAG.

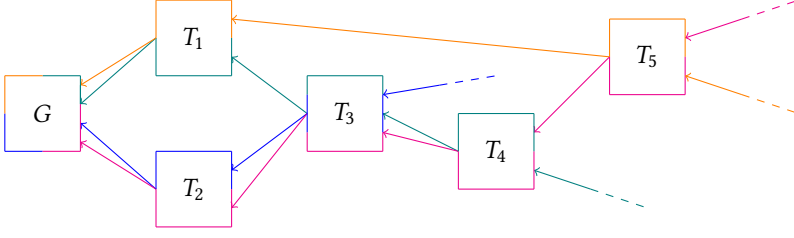


Fig. 2. The Multi-Linked DAG. The edge colours represent participants.

Multi-Linked DAG. Multi-Linked DAG (mDAG) of a validator v is a labelled directed graph $\mathcal{G}_v = (\mathbb{T}, E, \ell)$. The nodes of the graph $\mathbb{T}$ are tesseracts and the edges $E \subseteq \mathbb{T} \times \mathbb{T}$ are labeled by participants (given by $\ell : E \rightarrow \mathbb{P}$). A Multi-Linked DAG is given in Figure 2.

Tesseract Relationships. The Tesseracts form intuitive relationships driven by the edges of the mDAG. We say T is a p -parent of T' if there exists an edge $e = (T', T) \in E$ with the label $\ell(e) = p$. Conversely, T' is the p -child of T , and $T \preceq_p T'$ is the transitive closure of the p -parent relationship (T is a p -ancestor of T').

Intuitively, the parent-child relationship defines a partial order on the Tesseracts. The order is crucial to break ties between two competing Tesseracts which are updating the states of a common participant p . Formally, we say T conflicts with T' for a given participant $p \in P_T \cap P_{T'}$ if neither is a p -ancestor to the other.

A tesseract T corresponding to interaction I is said to *extend* a validator v 's state S if $T = \{\delta_I^p(S_p) \mid p \in P_I\}$. A tesseract is *committed* to the history of a validator by linking it with directed edges labeled by the participant to the tesseracts with the participant states of each of the interacting participants. More precisely, a tesseract T is committed to the history $\mathcal{G}_v = (\mathbb{T}, E, \ell)$ of validator v , resulting in an updated history $\mathcal{G}'_v = (\mathbb{T}', E', \ell')$ where

- (1) $\mathbb{T}' = \mathbb{T} \cup \{T\}$,
- (2) $E' = E \cup \{(T, T_p) \mid p \in P_T\}$, and
- (3) $\ell' = \ell \cup \{((T, T_p), p) \mid p \in P_T\}$.

Here, for each $p \in P_T$, T_p is the tesseract with the participant state S_p in $\mathcal{G}_v$. Thus, for each interacting participant p , the tesseracts with the earlier participant states are the p -parents of the committed tesseract.

Height. For a given Tesseract T , there exists many paths from it to the genesis Tesseract G in the global state. However, only one unique path for a given participant p exists i.e. the path with edges labelled p . Therefore, the height of a Tesseract is defined as a function of the participant. Formally, $T.\text{height}(p)$ is the length of the path following the edges labeled by p from the Tesseract T to the genesis Tesseract G .

Interaction Consensus Set. When a new interaction I is proposed, a set of validators communicate to reach consensus on the interaction. We define the set of validators as *Interaction Consensus Set* (ICS) for the interaction I and is denoted as $ICS(I)$.

The ICS consists of,

- (1) The context of each interacting participant $p \in P_I$, denoted as C_p .

- (2) A *stochastic set* of validators randomly sampled from the set of validators, denoted as S .
- (3) The *initiating* participant of the Interaction is allowed to specify a set of validators called the *preferential context* denoted as $\mathcal{P}_I$ that are to be included in the consensus process for the interaction.

The context of each participant $p \in P_I$ is sufficient to ensure that the protocol extends the latest state of each participant. However, the set is fixed and predetermined, and hence allows for bribery attacks [KKZ24]. Therefore, we include as a second component a variable *stochastic set* of validators randomly sampled from the set of validators. This ensures that the interaction is processed by a random subset of validators, preventing collusion and bribery attacks.

Quorum. A *quorum* is a set of validators that can reach consensus on an interaction. In the context of our protocol, a quorum is essential for ensuring that the interaction is agreed upon by a sufficient number of validators, thereby providing resilience against Byzantine faults.

In particular, a quorum consists of supermajorities from (1) the context of each interacting participant, (2) the stochastic set, and (3) the preferential context if present where a *supermajority* is a subset of $2f + 1$ unique validators from the set of validators X of size $n = 3f + 1$.

3 Model

We now describe the setting and the assumptions under which the protocol operates.

3.1 Network Model

3.1.1 Partial Synchrony. We model network communication as the partially synchronous model (introduced in [DLS88]) with a known bound Δ on message delay and an unknown instant called Global Stabilisation Time (GST), such that every message sent by an honest validator at time t to another honest validator arrives within time $\max(t, \text{GST}) + \Delta$.

3.1.2 Randomness. We assume the presence of a *distributed randomness beacon* that periodically provides random outputs to all validators.

3.2 Cryptographic Primitives

3.2.1 PKI and Digital Signatures. Each validator has a public-secret key pair. KRAMA uses BLS signatures [BLS01] with ideal security assumptions. All protocol messages are signed by their sender. We denote a message m from validator v as $\langle m \rangle_v$, or simply $\langle m \rangle$ when the sender is clear from context.

3.2.2 Verifiable Random Functions. Verifiable random functions (VRFs) [MVR99] take a public random seed and validator's secret key as input, generating a pseudo-random number and corresponding proof. Any validator can verify that the pseudo-random number corresponds to the random seed using the proof and the generator's public key. VRFs provide two key properties: (1) Pseudo-randomness—the output is indistinguishable from a uniformly random string to efficient adversaries without the secret key, and (2) Verifiability—anyone can verify that the pseudo-random number was generated by the validator holding the public key for the given seed using the proof given by the VRF. The VRF output is deterministic for a given seed and secret key. Hence, Byzantine validators cannot bias the randomness.

3.3 Adversary

A validator is *honest* if it follows the protocol and can send and receive messages as per the network model. A *Byzantine* validator deviates arbitrarily from the protocol but cannot forge honest validators' digital signatures or violate message delivery guarantees. We assume that every participant context and stochastic set with $n = 3f + 1$ validators has at most f Byzantine validators.

4 The Protocol

KRAMA protocol proceeds in a sequence of *epochs*, each lasting for a fixed duration. Within each epoch, validators operate in a *normal mode* where they reach consensus on Tesseract containing the latest state of interacting participants. Between epochs, validators enter a *reconfiguration mode* to update the validator list and prepare for the next epoch.

In this section, we first describe the normal operation of the protocol, detailing how validators reach consensus on Tesseracts within an epoch. Subsequently, we outline the reconfiguration mode, which occurs between epochs to update the validator list and prepare for the next epoch. At the beginning of each epoch, the validators are ordered globally and synchronized to the same view.

4.1 Normal mode

Each epoch is divided into a sequence of fixed-length *views*, where each view lasts for a duration of *viewLength*. Validators determine their current view using their local clock, referencing the time of the previous epoch transition. Validators receive interactions from participants and attempt to reach consensus on Tesseracts within each view.

Unlike other consensus protocols, KRAMA allows for multiple interactions to be executed and agreed upon within a single view, provided that the participant sets of these interactions are mutually disjoint. This means that validators can reach consensus on multiple Tesseracts in parallel, each involving different participants i.e. have multiple *consensus instances* running simultaneously. Note that executing multiple consensus instances in parallel is different from batching them together as a single block, as is done in traditional blockchain protocols. In KRAMA, each interaction results in a Tesseract that is agreed upon and committed independently.

4.1.1 Operators. Each *consensus instance*, involving an interaction I is driven by a designated operator O_I . KRAMA picks operators deterministically, thereby ensuring that all validators agree on the same operator. For example, one method would be to choose the operator from the context of the participants in a round-robin manner with a deterministic tie breaker among the participants.

Once an operator is chosen, each *consensus instance* proceeds in a sequence of four stages: Prepare, Propose, Vote, and Commit.

4.1.2 Prepare. As defined in Algorithm 1, the operator initiates the Prepare phase by sending a Prepare message to the *interaction's context*. This message contains the interaction and serves as a discovery mechanism for validators in the context to respond with their latest state information. We refer to the participant contexts of an interaction I together as the *interaction's context* denoted $\text{ctx}(I)$. In the Prepare phase, each validator (in the context of a participant p) collects Prepare messages from operators and until a timeout is reached, at which point it chooses a single Prepare message to respond to, ensuring that they are participating in a single consensus instance for participant p . This is accomplished by the `CHOOSE()` function. However, note that the validator

Algorithm 1 The Protocol: Stage 1 - Prepare

```

1: ▷ As an operator: ◀
2: upon view.begin do
3:   if  $\text{self} = o_I$  for interaction  $I$  then
4:      $I_v \leftarrow I$ 
5:     broadcast  $\langle \text{Prepare}, I_v \rangle$  to  $\text{CTX}(I_v)$ 

6: ▷ As a validator in a participant  $p$ 's context: ◀
7: upon view.begin do
8:    $\text{prepares}^p \leftarrow \{\}$ 

9: until prepareTimeout upon  $\langle \text{Prepare}, I \rangle_{op}$  do
10:   if  $op = o_I \wedge p \in P_I$  then
11:      $\text{prepares}^p \leftarrow \text{prepares}^p \cup \{\langle \text{Prepare}, I \rangle_{op}\}$ 

12: upon prepareTimeout do
13:    $\langle \text{Prepare}, I \rangle_{op} = \text{CHOOSE}(\text{prepares}_p)$ 
14:   send  $\langle \text{Prepare}, \{\text{locked}QC_v^p \mid p \in P_I\} \rangle$  to  $op$ 
15:   for all  $\langle \text{Prepare}, I' \rangle_{op'} \in \text{prepares}^p \setminus \{\text{CHOOSE}(\text{prepares}^p)\}$  do
16:     send  $\langle \text{PrepareNil}, \text{prepares}^p \rangle$  to  $op'$ 

```

may have responded to multiple Prepare messages for different participants in other consensus instances. For the remaining messages, validators respond with a PrepareNil message.

The operator collects responses from validators until a quorum of either Prepare or PrepareNil messages are obtained, at which point it can propose a Tesseract that extends the latest states of the participants from whose contexts a quorum of successful Prepare responses are obtained.

4.1.3 Propose. Algorithm 2 describes the Propose stage of the protocol. Among the responses to the Prepare messages obtained, the operator identifies the highest states and removes the outdated state information. This is accomplished by the function REMOVEOUTDATED(). The operator then proposes a Tesseract that extends these states and sends a Propose message by forming an ICS by sampling a stochastic set along with the interaction's context.

In the first view of the very first epoch, the operator may not have any previous state information to extend. In this case, the operator can propose a Tesseract that contains only the interaction itself, without extending any previous states.

Note that the extension is possible if there is no existing Tesseract that has been already agreed upon by the validators in prior views. If such a Tesseract exists, then the operator will re-propose the same Tesseract, ensuring that the consensus instance is consistent with the previous state. Further, the proposal is made only for Tesseracts with the same participant set from which the latest state information was obtained.

Validators accept the proposal only if it is *valid*, resulting from the extension of states after removing outdated ones.

Algorithm 2 The Protocol: Stage 2 - Propose

```

1: ▷ As an operator: ◀
2: upon quorum of Prepare or PrepareNil messages from  $\text{CTX}(ixn_v)$  do

```

```

3:    $P \leftarrow \{p \mid \text{received supermajority of Prepare messages from } C_p\}$ 
4:    $M \leftarrow \text{quorum of Prepare messages from } \bigcup_{p \in P} C_p$ 
5:    $\text{prepareQC} \leftarrow \text{QC}(M)$ 
6:    $\text{qcSet}_v \leftarrow \text{LOCKEDQCs}(\text{prepareQC})$ 
7:    $\text{REMOVEOUTDATED}(\text{qcSet}_v)$ 
8:   if  $\exists$  unique  $qc \in \text{qcSet}_v : qc.\text{stage} = \text{vote} \wedge qc.\text{participants} = P$  then
9:   |    $T \leftarrow \text{TESSERACT}(qc.\text{ixn})$ 
10:  else if  $\nexists qc \in \text{qcSet}_v : qc.\text{stage} = \text{vote}$  then
11:  |    $I' \leftarrow \text{EXTENDCOMMITTED}(\text{qcSet}, \mathcal{G}_v, P)$ 
12:  |    $T \leftarrow \text{TESSERACT}(I')$ 
13:  else
14:  |    $T \leftarrow \text{nil}$ 
15:   $\text{PROPOSE}(T, \text{prepareQC})$ 

```

16: **▷ As a validator in the ICS:** ◀

```

17: upon  $\langle \text{Propose}, T, \text{prepareQC}, \mathcal{S}, \text{rand}, \text{proof}, \text{view} \rangle$  from  $o_T$  do
18: |    $\text{qcSet}_v \leftarrow \text{UNIQUELOCKEDQCs}(\text{prepareQC})$ 
19: |   if  $T \neq \text{nil} \wedge \text{VALIDPROPOSE}(T, \text{qcSet}_v, \mathcal{S}, \text{rand}, \text{proof}, \text{view})$  then
20: |   |   send  $\langle \text{Vote}, T, \text{view}_v \rangle$  to  $\text{operator}_v$ 

```

```

21: upon  $\langle \text{Propose}, \text{nil}, \text{prepareQC}, \text{view} \rangle$  from  $op$  do
22: |   Terminate engaging in consensus for the interaction

```

23: **▷ Procedures:** ◀

```

24: procedure  $\text{PROPOSE}(T, \text{prepareQC})$ 
25: |   if  $T = \text{nil}$  then
26: |   |   broadcast  $\langle \text{Propose}, T, \text{prepareQC}, \text{view}_v \rangle$  to  $\text{CTX}(T)$ 
27: |   else
28: |   |    $(\mathcal{S}_v, \text{rand}_v, \text{proof}_v) \leftarrow \text{SELECTSTOCHASTIC}(T, \text{view}_v)$ 
29: |   |   broadcast  $\langle \text{Propose}, T, \text{prepareQC}, \mathcal{S}_v, \text{rand}_v, \text{proof}_v, \text{view}_v \rangle$  to  $\text{ICS}(T, \mathcal{S}_v)$ 

```

Algorithm 3 The Protocol: Stage 3 - Vote

1: **▷ As the interaction's operator:** ◀

```

2: upon quorum of Vote messages  $M$  from  $\text{ICS}_v$  do
3: |    $\text{voteQC} \leftarrow \text{QC}(M)$ 
4: |   broadcast  $\langle \text{Vote}, \text{tesseract}_v, \text{view}_v, \text{voteQC} \rangle$  to  $\text{ICS}_v$ 

```

5: **▷ As a validator in the ICS:** ◀

```

6: upon  $\langle \text{Vote}, \text{tesseract}_v, \text{view}_v, \text{voteQC} \rangle$  from  $\text{operator}_v$  do
7: |   for all  $p \in P_{\text{tesseract}_v}$  such that  $\text{self} \in C_p$  do
8: |   |    $\text{lockedQC}_v^p \leftarrow \text{voteQC}$ 
9: |   send  $\langle \text{Commit}, \text{tesseract}_v, \text{view}_v \rangle$  to  $\text{operator}_v$ 

```

4.1.4 Vote and Commit. The next two stages of the protocol are Vote and Commit phases, as described in Algorithm 3 and Algorithm 4 respectively. Validators that accept a proposal respond

Algorithm 4 The Protocol: Stage 4 - Commit

```

1: ▷ As the interaction's operator:
2: upon quorum of Commit messages  $M$  from  $ICS_v$  do
3:    $commitQC \leftarrow QC(M)$ 
4:   broadcast  $\langle Commit, tesseract_v, view_v, commitQC \rangle$  to  $ICS_v$ 
5:    $COMMIT(tesseract_v)$ 

6: ▷ As a validator in the ICS:
7: upon  $\langle Commit, tesseract_v, view_v, commitQC \rangle$  do
8:    $COMMIT(tesseract_v)$ 
9:   for all  $p \in P_{tesseract_v}$  such that  $self \in C_p$  do
10:     $lockedQC_v^p \leftarrow commitQC$ 

11: upon  $view.end$  do
12:   Terminate engaging in consensus for all interactions
  
```

with a Vote message and refrain from voting on other tesseracts with the same participant set. Once a quorum of validators accept the proposal, the operator forms a *quorum certificate* (QC) of the Vote responses and broadcasts it to the ICS. Validators on receiving this lock onto the Tesseract and pass it in any subsequent Prepare phases. This lock represents the validator's commitment to the proposed Tesseract. They further respond to the operator with a Commit message and the operator, upon receiving a quorum of such messages, forms a quorum certificate and sends a Commit message to the ICS as well as the entire validator set. Validators then commit the Tesseract to their histories and lock onto the QC.

4.2 Reconfiguration mode

At the beginning of each epoch, validators enter a *reconfiguration mode* to update the validator list and prepare for the next epoch. This reconfiguration process is crucial for maintaining the integrity and security of the protocol, as it allows validators to join or leave the network and ensures that the validator set remains up-to-date.

Two steps outline the reconfiguration process: validator list update and context shuffling. The former updates the list of validators for the next epoch, while the latter ensures that the contexts of the validators are shuffled to prevent collusion and bribery attacks.

4.2.1 Validator list update. During the epoch, validators may signal to the network their intent to leave or join. The requests are registered and updated in the local state at the beginning of the next epoch. Additionally, the ordering between validators are updated based on the latest validator list.

4.2.2 Context shuffling. The context of each participant will remain fixed for the duration of the epoch. However, at the beginning of each epoch, the context of each validator is shuffled to ensure that validators cannot collude with each other or engage in bribery attacks. The shuffling procedure follows the swap-or-not shuffle described in [HMR12], which is also detailed in the Ethereum 2.0 specifications and technical handbook [Edg]. This shuffle allows for the computation of the shuffled position of a single index in a list without requiring the entire list to be shuffled, making it efficient for the protocol's needs.

4.3 Utility methods

4.3.1 Operator selection. Deterministic selection of the operator for an interaction is crucial for ensuring that all validators agree on the same operator. This selection is based on the participant set of the interaction and the current view. The operator is chosen from the context of the participants in a round-robin manner, ensuring that each participant has a chance to drive consensus in different views.

Algorithm 5 Sampling the Stochastic Set

```

1: procedure SELECTSTOCHASTIC( $T, view$ )
2:    $V' \leftarrow V \setminus \bigcup_{p \in P_T} C_p$ 
3:    $seed \leftarrow \text{XOR}(\text{hash}(T), \text{RANDOM}(view))$ 
4:    $(rand, proof) \leftarrow \text{VRF}(seed, sk_{self})$ 
5:    $S \leftarrow \emptyset$ 
6:   for all  $i \in \{1, 2, \dots, n_S\}$  do
7:      $index \leftarrow \text{hash}(rand || i)$   $\triangleright index \in [1, n_V]$ 
8:      $S \leftarrow S \cup \{V'[index]\}$ 
9:   return  $(S, rand, proof)$ 

```

4.3.2 Stochastic set of validators. To decide on an interaction, the operator needs to form an ICS. A key component is the stochastic set of validators. The operator samples uniformly according to Algorithm 5 after excluding the validators part of a participant's context. The sample is associated with a proof obtained from a verifiable random function (VRF) to ensure that the sampling is verifiable by other validators.

4.3.3 Accepting the proposal for an interaction. Each validator that is part of a *consensus instance* receives a proposal from the operator and has to decide whether to accept or reject the proposal. The decision is based on the following criteria:

- (1) The validator belongs to the corresponding ICS formed with the interacting participants' contexts and the stochastic set.
- (2) The sender is the valid operator for the tesseract.
- (3) The validator did not Vote on a different tesseract with any of the interacting participants during the current view.
- (4) The proposed tesseract is the valid output of the state extension procedure undertaken by the operator.
- (5) The proposed stochastic set contains validators sampled according to the stochastic set selection procedure.

The logic for checking the criteria is abstracted in the function `VALIDPROPOSE()` in the pseudocode.

4.4 Blockchain Semantics

The protocol described above is agnostic to the underlying blockchain semantics. Here we describe the semantics of *interactions* and *Tesseracts* in the context of a blockchain.

4.4.1 Genesis tesseract. The genesis tesseract introduces a special participant SARGA (Sanskrit for 'genesis') that is responsible for maintaining the state of the blockchain. The context of SARGA is involved in initialising the context of new participants joining the system.

4.4.2 New participants. When a new participant p joins the system, its context is initialised via an interaction agreed upon by the context of SARGA. When the interaction is committed, the new participant is allowed to be part of further interactions.

4.4.3 Context updates between epochs. KRAMA pre-determines the context of each participant at the start of an epoch. However, when validators fail and no longer participant in *consensus instances*, the interactions of the corresponding participants may get blocked due to the lack of a quorum. To mitigate such scenarios, KRAMA allows for special interactions - *context updates* - on SARGA to update the context of a participant p in the middle of an epoch. This is done by creating a new interaction that updates the context of p in SARGA's state.

Validators when voting on *context updates*, additionally verify that marked validators are indeed faulty or unresponsive.

4.4.4 Quasi-permissionless. The reconfiguration protocol allows for validators to be added or removed from the system during epoch transitions, with the entire set of validators $\mathbb{V}$ in an epoch considered active. Therefore, KRAMA is characterized as a *quasi-permissionless* blockchain as per the definition in [LPR24].

5 Analysis

In this section, we describe and prove the safety and liveness properties of KRAMA.

To recall, KRAMA accepts interactions I (analogous to transactions) that result in Tesseract T (analogous to blocks) being committed by validators. We follow the traditional definitions of safety and liveness in the context of blockchains,

THEOREM 1 (SAFETY). *Between two conflicting Tesseracts T and T' , every honest validator will commit at most one of them.*

THEOREM 2 (LIVENESS). *In each view beginning after GST, in each maximal set of overlapping consensus instances with honest operators, at least one interaction is successfully committed by all honest validators within a bounded time τ .*

To show that KRAMA satisfies both Safety and Liveness, we rely on a key guarantee of the context shuffling and stochastic set sampling mechanisms used in the protocol.

ASSUMPTION 1 (RANDOM SAMPLING ASSUMPTION). *The stochastic set sampling and context shuffling mechanisms used in KRAMA ensure that with high probability, the sampled set of validators contains strictly less than one-third Byzantine validators.*

We satisfy the above assumption by using a hypergeometric distribution to sample a fixed-size set of validators from the total validator pool. We refer the reader to the Appendix for a detailed probabilistic analysis.

5.1 Safety

The analysis of safety of KRAMA is inspired by the analysis of Fast-Hotstuff's safety in [JNFG23].

Starting from the quorum intersection property, we first show that only one of two conflicting Tesseracts are voted for by honest validators. Then, we show that once a *supermajority* of validators vote for a Tesseract, it is guaranteed that the Tesseract will be committed in either this or later rounds therefore preserving safety.

LEMMA 1 (QUORUM INTERSECTION PROPERTY). *Let S be a set of $n = 3f + 1$ validators, with at most f Byzantine faults. For any two supermajorities over the set of validators S , we have at least one honest validator in common.*

LEMMA 2. *If there are two valid commitQCs for two conflicting tesseracts T and T' then $\text{VIEW}(T) \neq \text{VIEW}(T')$.*

PROOF. We prove by contradiction. Suppose, there exist two QCs *commitQC* and *commitQC'* for the tesseracts T and T' formed during the same view. Since the tesseracts T and T' are conflicting, there exists a participant $p \in P_T \cap P_{T'}$. Therefore, by definition of a Tesseract's ICS, $C_p \subseteq \text{ICS}(T)$ as well as $C_p \subseteq \text{ICS}(T')$. Now, by Lemma 1, we have an honest validator $v \in C_p$ that is a part of both QCs. However, honest validators do not vote for multiple tesseracts involving the same participant. Hence, the result follows by contradiction. $\square$

LEMMA 3. *If an honest validator has received precommitQC for tesseract T then the corresponding prevoteQC will be a lockedQC part of the Prepare messages obtained by the operator in the next consensus instance for each participant $p \in P_T$.*

PROOF. If an honest validator received a *precommitQC*, then the validators whose PreCommit messages comprise the quorum certificate have received *prevoteQCs*. Therefore, a quorum of validators are locked onto the *prevoteQC*. Now, consider the next view with a consensus instance containing a participant $p \in P_T$. During the Prepare stage, say the operator obtains a supermajority of Prepare messages from p 's context. Note that the quorum of validators locked onto the *prevoteQC* consists of a supermajority of votes from p 's context. Therefore, by Lemma 1 there exists an honest validator $v \in C_p$ that is present in both of the above supermajorities. This validator sends the *prevoteQC* as its locked QC in the Prepare message and is therefore obtained by the operator. $\square$

Thus, tesseracts previously committed by honest validators are learned by operators during the prepare stage at the start of each view.

THEOREM 3. *Two conflicting tesseracts cannot both be committed by honest validators.*

PROOF. An honest validator commits a tesseract only upon receiving a valid *precommitQC*. From Lemma 2, it follows that the conflicting tesseracts' *precommitQCs* were formed in different views. Thus, without loss of generality, consider two conflicting tesseracts T and T' with a participant $p \in P_T \cap P_{T'}$ where $\text{VIEW}(T) < \text{VIEW}(T')$. For tesseract T' to conflict with T , it has to extend an ancestor of T . However, by Lemma 3, we know that the operator for the subsequent view with an interaction for participant p after $\text{VIEW}(T)$ will have the *prevoteQC* formed for tesseract T as part of its *prepareQC*. The operator, while determining which interaction to propose, thus considers the above *prevoteQC* as well. In this procedure by the operator, there are two outcomes for this QC: it can be removed from the set of locked QCs or it can stay as part of the set. If it is not removed, then the proposal has to be a p -child to tesseract T based on the protocol rules that proposals must satisfy. If it is removed, then the proposal has to extend the latest state of each of the interacting participants, and therefore it will be a p -child of the committed tesseract T . Thus in both cases the resulting proposal is a p -child of T and therefore T' cannot extend an ancestor of T . Moreover, operators cannot forge a *prepareQC* for T' since validators verify the quorum certificates. Hence, the result follows. $\square$

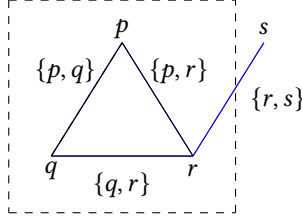


Fig. 3. A maximal set of overlapping consensus instances. Vertices represent participants and edges represent consensus instances. Here, the set of black consensus instances is not maximal.

5.2 Liveness

A key novelty of KRAMA is the use of consensus instances that are updating disconnected parts of the global states - or participant sets. Therefore, a global liveness property should ensure first that among the many overlapping consensus instances, at least one succeeds in committing a Tesseract.

To illustrate, consider a set of overlapping consensus instances, each with a different set of participants. As described in Fig. 3, we have 4 concurrent consensus instances with participants p, q, r , and s - $\{p, q\}$, $\{q, r\}$, $\{p, r\}$, and $\{r, s\}$. Here, the consensus instances $\{p, q\}$, $\{q, r\}$, and $\{p, r\}$ overlap with each other. Yet, KRAMA cannot ensure that among these set of overlapping consensus instances, at least one instance will successfully commit. Since the validators associated with p, q , and r can choose to respond to the consensus instance $\{p, q\}$, $\{q, r\}$, or $\{p, r\}$, they may not all agree on a single instance required to achieve a quorum.

However, the maximal set of overlapping consensus instances is $\{p, q\}$, $\{q, r\}$, $\{p, r\}$, and $\{r, s\}$, as the instance $\{r, s\}$ overlaps with $\{q, r\}$ and $\{p, r\}$. In this case, KRAMA ensures that at least one of the consensus instances in the maximal set will successfully commit a Tesseract. Recall that in the Prepare phase of KRAMA, validators pick a prepare message to respond from the set of overlapping consensus instances using the `CHOOSE()` function. By ensuring that the `CHOOSE()` function deterministically orders the consensus instances and picks the highest one among them, we can guarantee that at least one consensus instance in a maximal set of overlapping consensus instances will be chosen by all participants in the context of the participants in the maximal set.

Formally, we define overlapping consensus instances and maximal sets of overlapping consensus instances as follows.

DEFINITION 1 (OVERLAPPING CONSENSUS INSTANCE). *A consensus instance is said to be overlapping with another if they share at least one participant in their participant sets.*

DEFINITION 2 (MAXIMAL SET OF OVERLAPPING CONSENSUS INSTANCES). *A maximal set of overlapping consensus instances is a set of consensus instances such that every instance in the set overlaps with at least one other instance in the set, and no additional instance can be added to the set without violating the overlap condition.*

Prior to proving liveness, we first show that in each maximal set of overlapping consensus instances with honest operators, at least one consensus instance successfully obtains a quorum of Prepare messages from each of its participants' contexts. Subsequently, we use the lemma to show that at least one interaction among a maximal set will eventually be successfully committed by all honest validators.

LEMMA 4. *In each maximal set of overlapping consensus instances with honest operators, at least one consensus instance successfully obtains a quorum of Prepare messages from each of its participants' contexts.*

PROOF. Consider a view after GST with a maximal set of overlapping consensus instances led by honest operators. Let P be the union of all the participant sets in the consensus instances. Validators in each context C_p for $p \in P$, choose a single consensus instance from the Prepare messages they have received. Let $prepares_p$ be the set of Prepare messages received by validators in C_p . Since the operators are honest, all validators in a participant's context receive the same set of Prepare messages as operators broadcast them to the entire context. Validators make the choice of a consensus instance to respond to by ordering the Prepare messages that they have received. This order is based on view randomness and is thus common to all context validators for participants in P . Consider the consensus instance that is the maximum element in the order. Each of its participant set context validators will choose it as it is the highest among the Prepare messages they have received as it is the maximum element. Further, since the set of overlapping consensus instances is maximal, there is no other consensus instance that intersects with any of the instances in the given set. Therefore, the consensus instance that is maximum in the order successfully obtains a quorum. $\square$

Algorithm 6 Removing outdated locks

```

1: procedure REMOVEOUTDATEDQCs( $qcSet$ )
2:   COMPUTEHIGHQCs( $qcSet$ )
3:   for all  $qc \in qcSet$  such that  $qc.stage = vote$  do
4:     for all  $p \in qc.participants$  do
5:       if  $qc.height(p) \leq highCommitQC_p.height(p) \vee qc.height(p) <$ 
         $highVoteQC_p.height(p)$  then
6:          $qcSet \leftarrow qcSet \setminus \{qc\}$ 
7:         break
8:       else if  $qc.height(p) = highVoteQC_p.height(p) \wedge qc.view < highVoteQC_p.view$  then
9:          $qcSet \leftarrow qcSet \setminus \{qc\}$ 
10:      break
  
```

Additionally, during the Prepare stage for a given participant set P , Byzantine validators can send outdated Vote locks in whose place other tesseracts have been committed in participants not in P to prevent the operator from making a proposal due to the presence of more than one Vote locked tesseract. Therefore, it is crucial for liveness that such outdated information is removed from the locks obtained. This is accomplished by the REMOVEOUTDATED() procedure given in Algorithm 6 that we describe below. The outdated locks are removed by comparing with the maximum height locks obtained for each participant present in the locked tesseracts. The outdated vote QCs are filtered by removing those where:

- (1) A participant p in the QC has a commit QC with atleast the same height for the participant
- (2) A participant p in the QC has a vote QC higher for the participant

This ensures that vote locks that have been superseded by committed tesseracts are ignored.

THEOREM 4. *In each view beginning after GST, in each maximal set of overlapping consensus instances with honest operators, at least one interaction is successfully committed by all honest validators within a bounded time T .*

PROOF. By Lemma 4, in the given scenario, at least one consensus instance obtains a quorum of Prepare messages and therefore proceeds to make a proposal. The outdated locked QCs received in the quorum of Prepare messages are removed by the operator using the `REMOVEOUTDATEDQCs()` procedure. This ensures that Byzantine validators cannot prevent proposals being made by sending outdated locks for tesseract with different participant sets. Therefore, a tesseract extending the latest locks is proposed as per the `EXTENDCOMMITTED()` function ensuring safety. During this view, validators in the ICS can successfully commit the proposal as supermajorities of Vote and Commit messages are formed during the respective stages, since message delay is bounded by Δ and the length of the view is assumed to be sufficiently long for the voting rounds' communication to occur between the operator and validators. Here, supermajority of votes are formed as validators send Vote messages if the proposed tesseract is the result of the proposal determining procedure run by the operator, as given in Section 4.3.3. Validators not in the ICS commit the tesseract upon receiving it from the operator. $\square$

6 Related Work

Recent years have witnessed significant advancements in distributed ledger technologies, particularly in the design of consensus protocols and scalable blockchain architectures. Early systems such as Bitcoin [Nak09] introduced decentralized payment networks, while Ethereum [But15] extended this vision to general-purpose smart contracts, enabling a broader range of decentralized applications. As the adoption of blockchains has grown, so has the need for scalable, secure, and decentralized consensus mechanisms.

Consensus protocols in blockchains can be broadly categorized into longest-chain protocols, such as those in the Ouroboros family [KRDO17, DGKR18, BGK⁺18] and Snow White [DPS19], and protocols inspired by classical Byzantine Fault Tolerance (BFT) research [Bra87, LSP82, CL99]. The latter category includes modern protocols like Tendermint [BKM18], Algorand [GHM⁺17], Hotstuff [YMR⁺19], and Fast-Hotstuff [JNFG23], primarily motivated by PBFT [CL99], which leverage round-based consensus with rotating leaders to improve performance and robustness. Avalanche [RYS⁺20] represents a distinct approach, employing metastable consensus through repeated sampling without a designated leader. Comprehensive surveys such as [BSAB⁺19] provide an overview of these developments.

AptosBFT [Lab22a] and Sui [Lab22b] are recent protocols that extend the Hotstuff family, focusing on high throughput and low-latency consensus by leveraging parallel execution and advanced mempool designs. Both systems employ a global consensus protocol to order transactions and achieve scalability by decoupling execution from consensus, allowing for efficient processing of independent transactions. In contrast, KRAMA departs from the global ordering paradigm by associating a set of validators with each participant and maintaining a separate chain per participant. Rather than relying on a single global consensus instance, KRAMA commits multi-participant transactions across the relevant chains, enabling parallelism without requiring a total order on all transactions.

Committee-based consensus protocols have emerged as a means to enhance scalability and efficiency. In these systems, a subset of validators (the committee) is selected to participate in consensus for a given period or instance [ZMR18, GHM⁺17, PS17]. Dynamic committee formation, as adopted

in our protocol, allows multiple consensus instances to proceed in parallel, further improving throughput.

Sharding is another prominent technique for scaling blockchains, wherein the network is partitioned into multiple shards, each processing a subset of transactions independently [KKJG⁺17, SP19, Tea17, Har20]. While sharding increases throughput, it introduces challenges in cross-shard communication and state management. In contrast, our approach associates a set of validators with each participant, maintaining a separate chain per participant and committing multi-participant transactions across relevant chains. Unlike state sharding, all validators in our protocol maintain the entire system state, simplifying consistency and security guarantees.

Finally, while Krama employs a multi-linked Directed Acyclic Graph (DAG) structure to record validator states, it differs fundamentally from DAG-based consensus protocols such as DAG-Rider [KKKS21], Bullshark [SGSKK22], and Cordial Miners [KNPS22], which decouple block dissemination from total ordering. In Krama, the DAG serves as a local log of states to be agreed upon, rather than as a mechanism for achieving consensus itself.

7 Conclusion

References

- [BGK⁺18] Christian Badertscher, Peter Gazi, Aggelos Kiayias, Alexander Russell, and Vassilis Zikas. Ouroboros genesis: Composable proof-of-stake blockchains with dynamic availability. *Cryptology ePrint Archive*, Paper 2018/378, 2018.
- [BKM18] Ethan Buchman, Jae Kwon, and Zarko Milosevic. The latest gossip on bft consensus, 2018.
- [BLS01] Dan Boneh, Ben Lynn, and Hovav Shacham. Short signatures from the weil pairing. In Colin Boyd, editor, *Advances in Cryptology – ASIACRYPT 2001*, pages 514–532, Berlin, Heidelberg, 2001. Springer Berlin Heidelberg.
- [Bra87] Gabriel Bracha. Asynchronous byzantine agreement protocols. *Information and Computation*, 75(2):130–143, 1987.
- [BSAB⁺19] Shehar Bano, Alberto Sonnino, Mustafa Al-Bassam, Sarah Azouvi, Patrick McCorry, Sarah Meiklejohn, and George Danezis. Sok: Consensus in the age of blockchains. In *Proceedings of the 1st ACM Conference on Advances in Financial Technologies*, AFT ’19, page 183–198, New York, NY, USA, 2019. Association for Computing Machinery.
- [But15] Vitalik Buterin. A next-generation smart contract and decentralized application platform. 2015.
- [CL99] Miguel Castro and Barbara Liskov. Practical byzantine fault tolerance. In *Proceedings of the Third Symposium on Operating Systems Design and Implementation*, OSDI ’99, page 173–186, USA, 1999. USENIX Association.
- [DGKR18] Bernardo David, Peter Gazi, Aggelos Kiayias, and Alexander Russell. Ouroboros praos: An adaptively-secure, semi-synchronous proof-of-stake blockchain. In Jesper Buus Nielsen and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2018*, pages 66–98, Cham, 2018. Springer International Publishing.
- [DLS88] Cynthia Dwork, Nancy Lynch, and Larry Stockmeyer. Consensus in the presence of partial synchrony. *J. ACM*, 35(2):288–323, apr 1988.
- [DPS19] Phil Daian, Rafael Pass, and Elaine Shi. Snow white: Robustly reconfigurable consensus and applications to provably secure proof of stake. In Ian Goldberg and Tyler Moore, editors, *Financial Cryptography and Data Security*, pages 23–41, Cham, 2019. Springer International Publishing.
- [Edg] Ben Edgington. Upgrading ethereum. <https://eth2book.info/>. Accessed: 2025-7-1.
- [GHM⁺17] Yossi Gilad, Rotem Hemo, Silvio Micali, Georgios Vlachos, and Nickolai Zeldovich. Algorand: Scaling byzantine agreements for cryptocurrencies. In *Proceedings of the 26th Symposium on Operating Systems Principles*, SOSP ’17, page 51–68, New York, NY, USA, 2017. Association for Computing Machinery.
- [Har20] Team Harmony. Harmony: Technical whitepaper, 2020.
- [HMR12] Viet Tung Hoang, Ben Morris, and Phillip Rogaway. An enciphering scheme based on a card shuffle. In Reihaneh Safavi-Naini and Ran Canetti, editors, *Advances in Cryptology – CRYPTO 2012*, pages 1–13, Berlin, Heidelberg, 2012. Springer Berlin Heidelberg.
- [JNFG23] Mohammad M. Jalalzai, Jianyu Niu, Chen Feng, and Fangyu Gai. Fast-hotstuff: A fast and robust bft protocol for blockchains. *IEEE Trans. Dependable Secur. Comput.*, 21(4):2478–2493, August 2023.

- [KKJG⁺17] Eleftherios Kokoris-Kogias, Philipp Jovanovic, Linus Gasser, Nicolas Gailly, Ewa Syta, and Bryan Ford. OmniLedger: A secure, scale-out, decentralized ledger via sharding. *Cryptology ePrint Archive*, Paper 2017/406, 2017.
- [KKKNS21] Idit Keidar, Eleftherios Kokoris-Kogias, Oded Naor, and Alexander Spiegelman. All you need is dag. In *Proceedings of the 2021 ACM Symposium on Principles of Distributed Computing*, PODC’21, page 165–175, New York, NY, USA, 2021. Association for Computing Machinery.
- [KKZ24] Dimitris Karakostas, Aggelos Kiayias, and Thomas Zacharias. Blockchain bribing attacks and the efficacy of counterincentives. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, CCS ’24, page 1031–1045, New York, NY, USA, 2024. Association for Computing Machinery.
- [KNPS22] Idit Keidar, Oded Naor, Ouri Poupko, and Ehud Shapiro. Cordial miners: Fast and efficient consensus for every eventuality. *Schloss Dagstuhl – Leibniz-Zentrum für Informatik*, 2022.
- [KRDO17] Aggelos Kiayias, Alexander Russell, Bernardo David, and Roman Oliynykov. Ouroboros: A provably secure proof-of-stake blockchain protocol. In Jonathan Katz and Hovav Shacham, editors, *Advances in Cryptology – CRYPTO 2017*, pages 357–388, Cham, 2017. Springer International Publishing.
- [Lab22a] Aptos Labs. The aptos blockchain: Safe, scalable, and upgradeable web3 infrastructure. https://aptosfoundation.org/whitepaper/aptos-whitepaper_en.pdf, 2022. Accessed: 2024-06-01.
- [Lab22b] Mysten Labs. The sui smart contracts platform. <https://docs.sui.io/paper/sui.pdf>, 2022. Accessed: 2024-06-01.
- [LPR24] Andrew Lewis-Pye and Tim Roughgarden. Permissionless consensus, 2024.
- [LSP82] Leslie Lamport, Robert Shostak, and Marshall Pease. The byzantine generals problem. *ACM Trans. Program. Lang. Syst.*, 4(3):382–401, jul 1982.
- [MVR99] Silvio Micali, Salil Vadhan, and Michael Rabin. Verifiable random functions. In *Proceedings of the 40th Annual Symposium on Foundations of Computer Science*, FOCS ’99, page 120, USA, 1999. IEEE Computer Society.
- [Nak09] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system. *Cryptography Mailing list at https://metzdowd.com*, 03 2009.
- [PS17] Rafael Pass and Elaine Shi. Hybrid Consensus: Efficient Consensus in the Permissionless Model. In Andréa Richa, editor, *31st International Symposium on Distributed Computing (DISC 2017)*, volume 91 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 39:1–39:16, Dagstuhl, Germany, 2017. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
- [RYS⁺20] Team Rocket, Maofan Yin, Kevin Sekniqi, Robbert van Renesse, and Emin Gün Sirer. Scalable and probabilistic leaderless bft consensus through metastability, 2020.
- [SGSKK22] Alexander Spiegelman, Neil Giridharan, Alberto Sonnino, and Lefteris Kokoris-Kogias. Bullshark: Dag bft protocols made practical. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, CCS ’22, page 2705–2718, New York, NY, USA, 2022. Association for Computing Machinery.
- [SP19] Alex Skidano and Illia Polosukhin. Nightshade: Near protocol sharding design 2.0, 2019.
- [Tea17] The ZILLIQA Team. The zilliqa technical whitepaper, 2017.
- [YMR⁺19] Maofan Yin, Dahlia Malkhi, Michael K. Reiter, Guy Golan Gueta, and Ittai Abraham. Hotstuff: Bft consensus with linearity and responsiveness. In *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing*, PODC ’19, page 347–356, New York, NY, USA, 2019. Association for Computing Machinery.
- [ZMR18] Mahdi Zamani, Mahnush Movahedi, and Mariana Raykova. Rapidchain: Scaling blockchain via full sharding. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, CCS ’18, page 931–948, New York, NY, USA, 2018. Association for Computing Machinery.